

1. (1) 我曾经有过多人协同开发的经历，小组成员分别完成分配好的任务，实现各自的功能，最后把代码合并起来。

(2) 日常的工作和开发中，对于某个代码可能需要经常修改，先暂存再提交是为了最后敲定哪些修改是需要提交的，哪些是不需要提交到最终结果的。

(3) `git branch` 和 `git branch -a` 的区别在于前者只显示本地的分支 (`branch`)，而后者会同时显示本地分支和远程跟踪的分支。

2. 修改为：增设一行 `printf("Hello, ICS!");`

3. (1) 有利于团队协作。一个大项目几乎很难通过仅仅一个人的努力就能够完成，这就需要我们使用 `git` 这样的工具来进行团队协作；

(2) 有利于管理和追溯版本改动。语义化版本告诉我们，版本号不是随便递增的数字，而是对使用者的承诺：这个改动是否兼容、是否安全升级。而 `Git` 正是提交历史的载体，学会 `Git`，我才能把"我改了什么、为什么改、影响多大"清晰地记录下来，而不是留下一堆无法回溯的混乱改动。

4.

```
git merge feature
Already on 'main'
Your branch is up to date with 'origin/main'.
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

这说明 `main` 分支和 `feature` 分支在合并的时候发生了冲突。

```

● (base) root@LAPTOP-309DR96H:/home/xiaowu/计算机系统基础/ICS_Lab0# git log --one
line --graph --all
*   ebef0da (HEAD -> main, origin/main, origin/HEAD) Merge branch 'feature'
| \
|  * 1e80e41 (origin/feature, feature) feat:在feature分支上修改main.c
* | f116016      modified:    main.c
|/
* f6c0a26      modified:    main.c
* e17e4f2 Initial commit

```

通过同时保留 main 分支和 feature 分支的修改，最终合并成功。

5.实验步骤

- (1) 创建代码仓库拉取代码；
- (2) 修改 main 分支中的 main.c：增设一行 `printf("Hello, ICS!");`;
- (3) 创建 feature 分支，并修改 main.c：增设一行 `printf("Hello, feature branch!");`;
- (4) 修改 main 分支中的 main.c：将 `printf("Hello, ICS!");` 改为 `printf("Hello, main branch!");`;
- (5) 合并分支，产生冲突时，同时保留 (3) (4) 的修改，解决冲突。